



Natural Language Processing

Lecture 13: Deployment. Performance. Quantization.

Acknowledgement

Some slides were adapted from Arthur Douillard's talk "Distributed Training in Machine Learning". [Check it out!](#)

Agenda

1. Parallelism
2. Compression
3. Inference

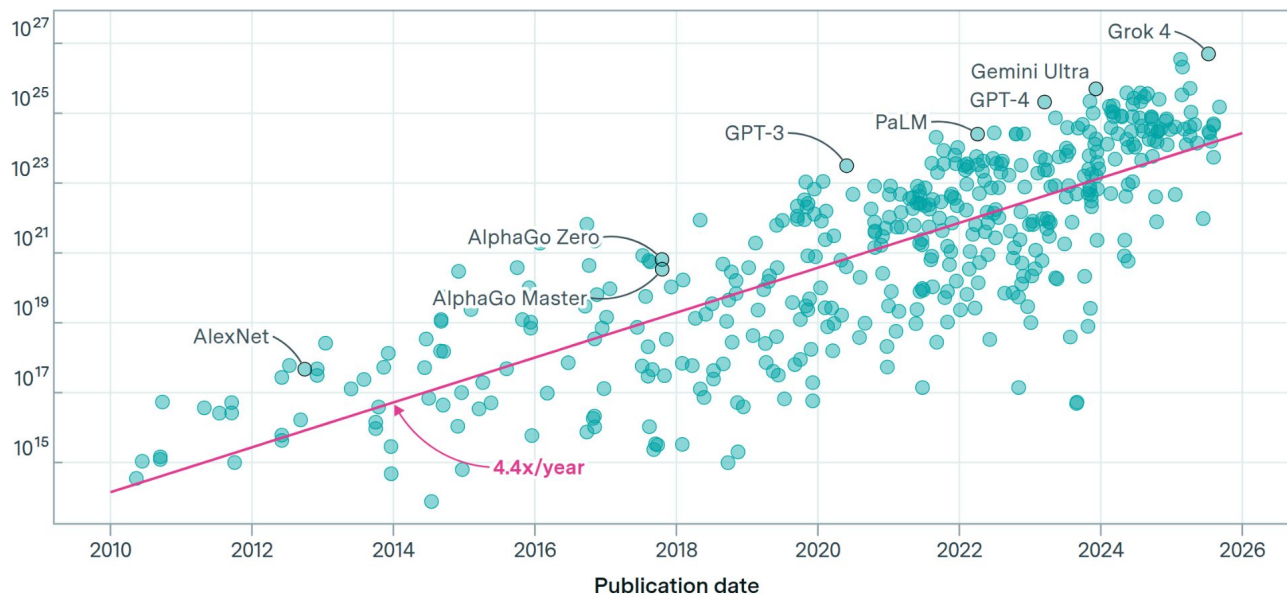
The Bitter Lesson

Training compute of notable models

EPOCH AI

Training compute (FLOP)

443 models



CC-BY

epoch.ai

Q: How can we improve “AI”?

Q: How can we improve “AI”?

1. Algorithmic innovations
2. More data
3. Bigger models

A batch size is too big?

Chinchilla scaling laws stated that that for each neuron in your model you needed 20 tokens, a 20:1 ratio. So to train a 10B parameters model, you need 200B tokens.

But people overtrain their model much more, Llama3 405B was trained on 15.6T tokens, a 38:1 ratio. It's even more true for small LM, Smollm3 is a 3B parameters model trained on 11T tokens, a 3666:1 ratio!

Smollm3 model can fit easily in a single GPU, but if you plan to train 11T tokens with a single GPU (and thus a small batch size), it'll take you decades!

A too large batch size, and you cannot fit the activations & gradients in the unique GPU.

There are really big models

Llama 4 Behemoth has 2 **trillion** total parameters!

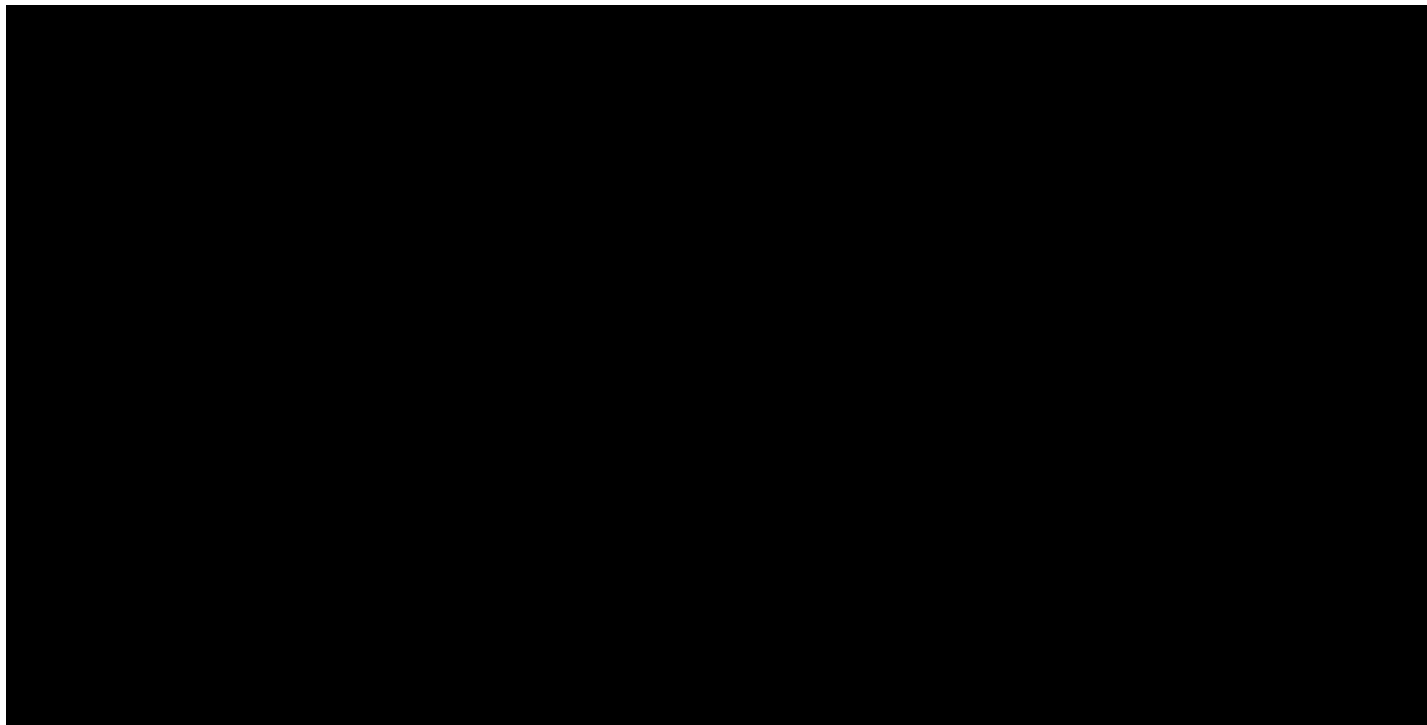
Stored in Float32, that's $2e12 * 4 / 1e9 = 8,000$ GB

H200 has 141GB of VRAM, you cannot fit the model parameters on a single GPU.
Also, you need even more space for the activations and gradients (+ 16,000 GB)!

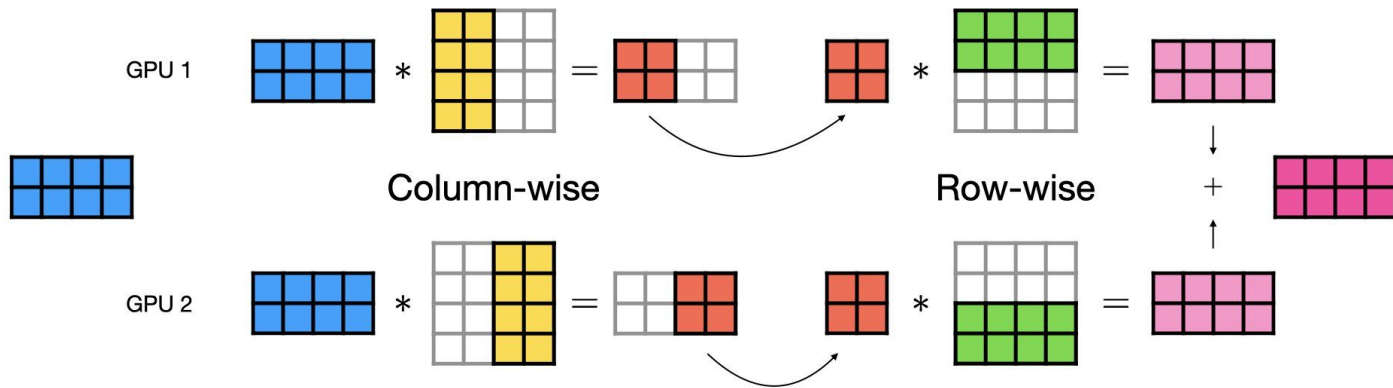
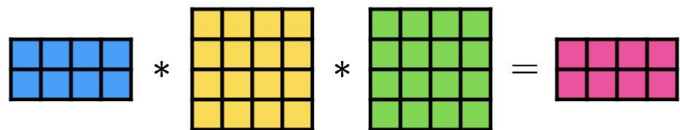
Q: What can we do with cluster of several GPUs?

Parallelism

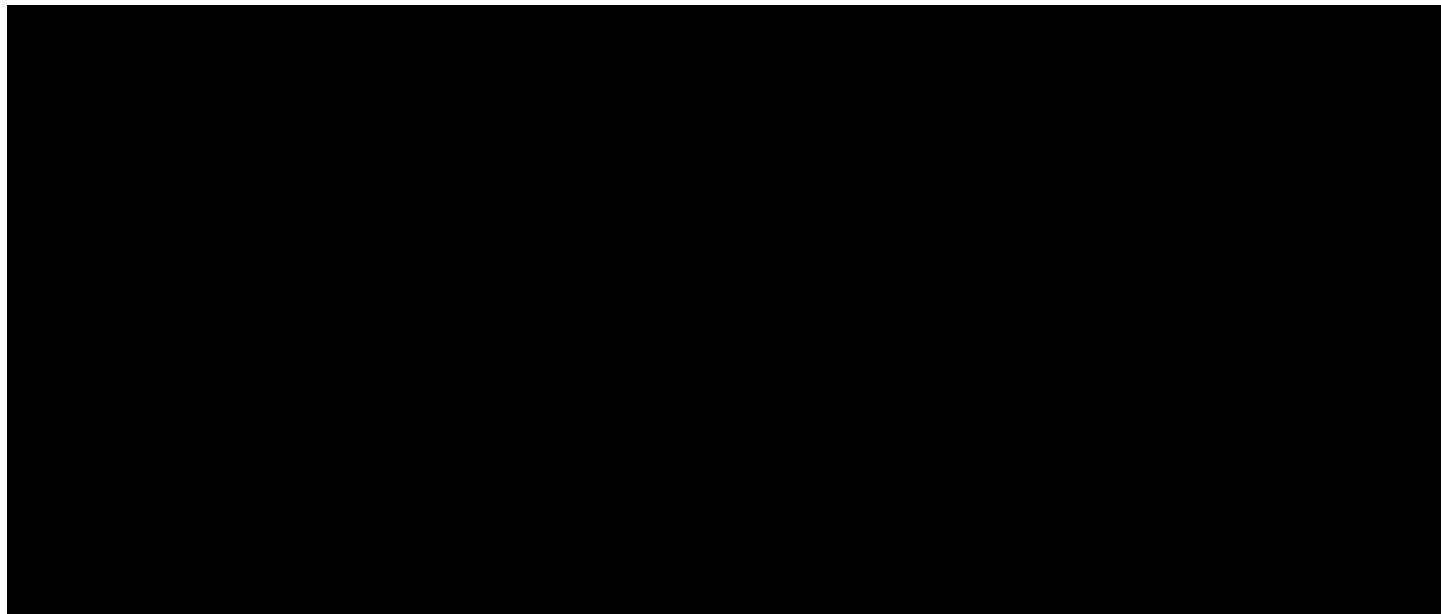
Distributed Data Parallelism



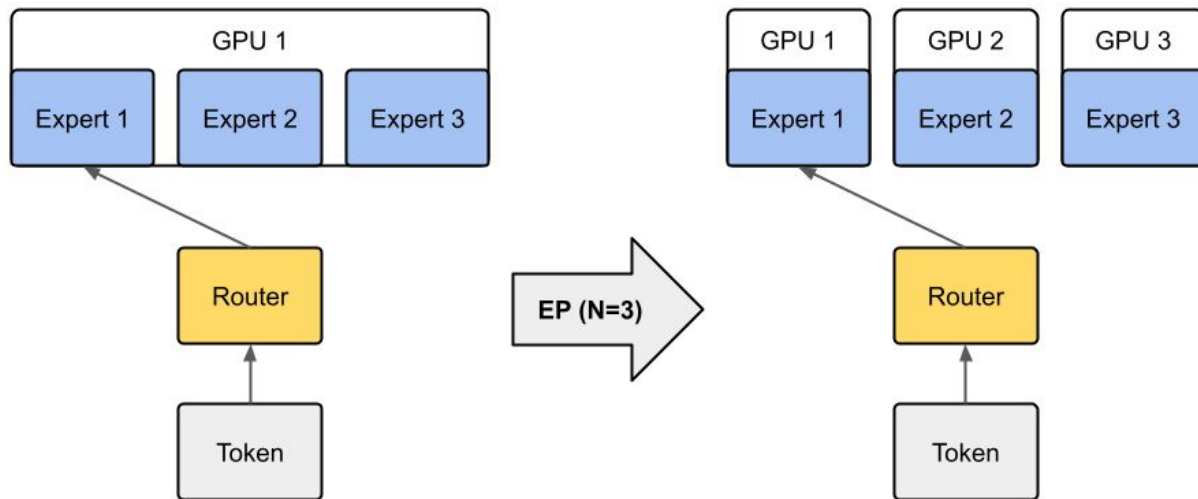
Tensor Parallelism



Pipeline Parallelism



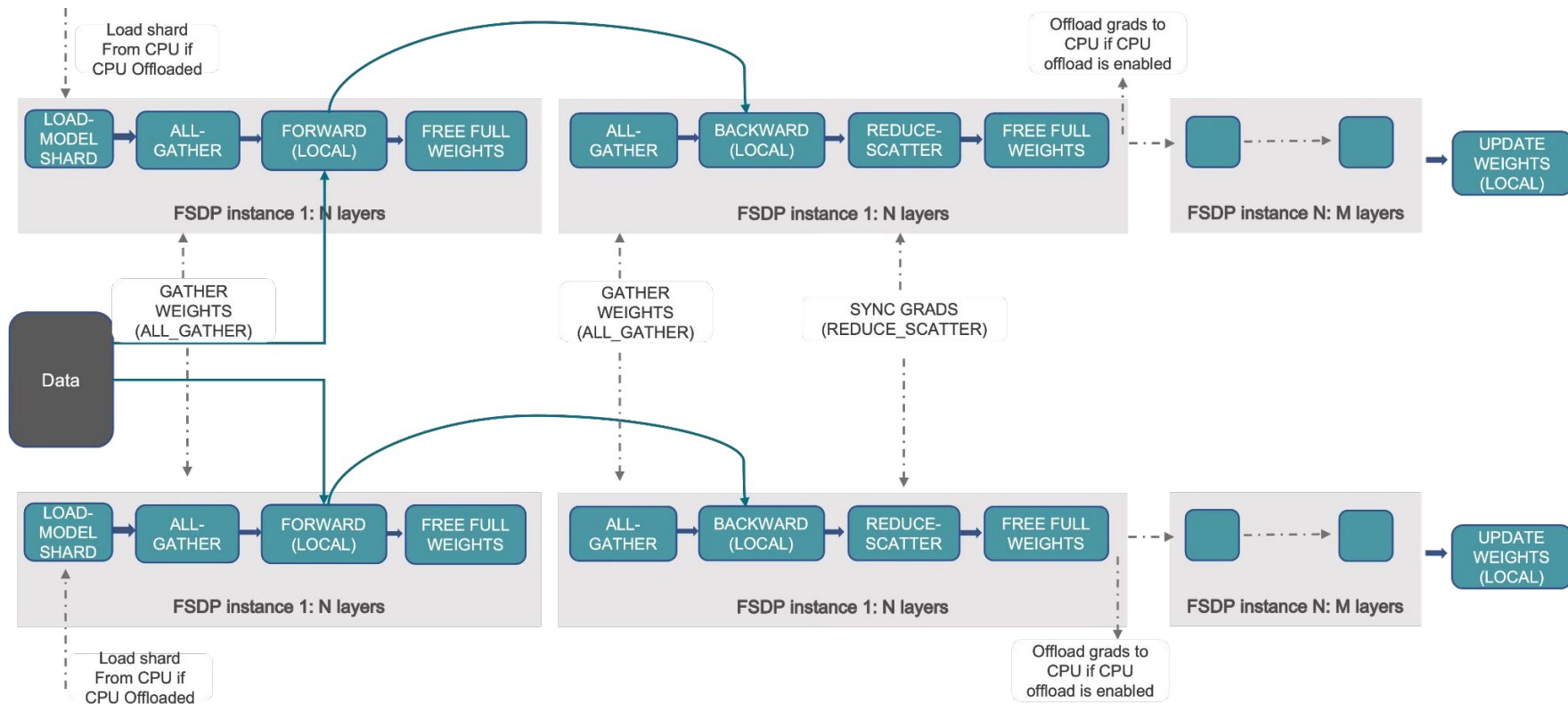
Expert Parallelism



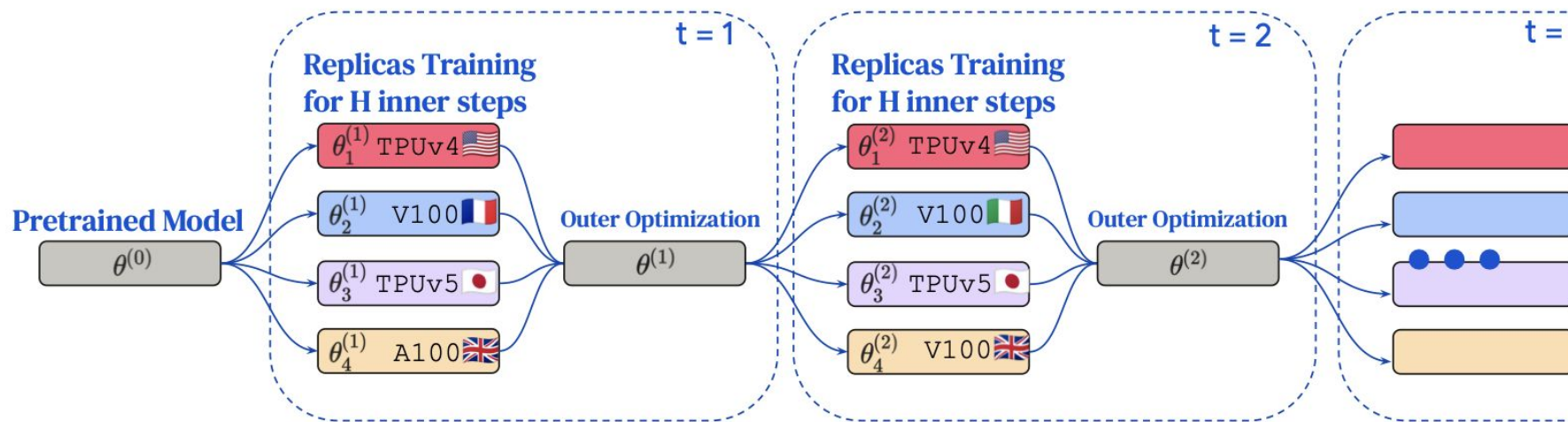
Activation Partitioning

Feature	Sequence Parallelism (SP)	Context Parallelism (CP)
Motivation	Reduce activation memory	Train extremely long sequences
Sharding dimension	Sequence length (activations)	Sequence length (attention context)
Attention algorithm	Unchanged	Modified (ring / block attention)
Does each GPU need full sequence for attention?	Yes (all-gather)	No
Communication pattern	all-gather + reduce-scatter	streaming Q/K/V exchange, ring attention
Scales to sequence length	~8k–32k	32k–1M+
Compatibility	Works with tensor & pipeline parallel	More complex integration

Fully Sharded Data Parallel (FSDP)



Distributed Low-Communication (DiLoCo)



Distributed Low-Communication (DiLoCo)

Algorithm 1 DiLoCo Algorithm

Require: Initial model $\theta^{(0)}$

Require: k workers

Require: Data shards $\{\mathcal{D}_1, \dots, \mathcal{D}_k\}$

Require: Optimizers InnerOpt and OuterOpt

```
1: for outer step  $t = 1 \dots T$  do
2:   for worker  $i = 1 \dots k$  do
3:      $\theta_i^{(t)} \leftarrow \theta^{(t-1)}$ 
4:     for inner step  $h = 1 \dots H$  do
5:        $x \sim \mathcal{D}_i$ 
6:        $\mathcal{L} \leftarrow f(x, \theta_i^{(t)})$ 
7:        $\triangleright$  Inner optimization:
8:        $\theta_i^{(t)} \leftarrow \text{InnerOpt}(\theta_i^{(t)}, \nabla \mathcal{L})$ 
9:     end for
10:   end for
11:    $\triangleright$  Averaging outer gradients:
12:    $\Delta^{(t)} \leftarrow \frac{1}{k} \sum_{i=1}^k (\theta^{(t-1)} - \theta_i^{(t)})$ 
13:    $\triangleright$  Outer optimization:
14:    $\theta^{(t)} \leftarrow \text{OuterOpt}(\theta^{(t-1)}, \Delta^{(t)})$ 
15: end for
```

Q: What can you do if you have to use only one GPU?

Compression

Universal Approximation Theorem

Universal approximation theorem—Let $C(X, \mathbb{R}^m)$ denote the set of **continuous functions** from a subset X of a Euclidean $\mathbb{R}^n$ space to a Euclidean space $\mathbb{R}^m$. Let $\sigma \in C(\mathbb{R}, \mathbb{R})$. Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x .

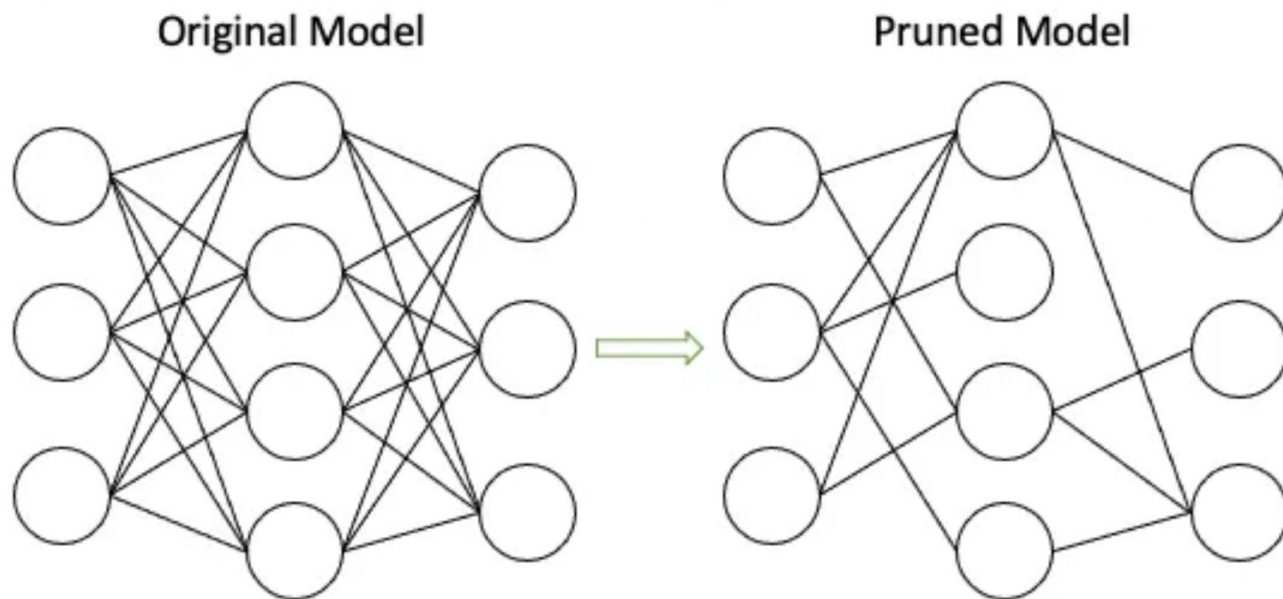
Then σ is not **polynomial if and only if** for every $n \in \mathbb{N}$, $m \in \mathbb{N}$, **compact** $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m)$, $\varepsilon > 0$ there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

$$\sup_{x \in K} \|f(x) - g(x)\| < \varepsilon$$

where $g(x) = C \cdot (\sigma \circ (A \cdot x + b))$

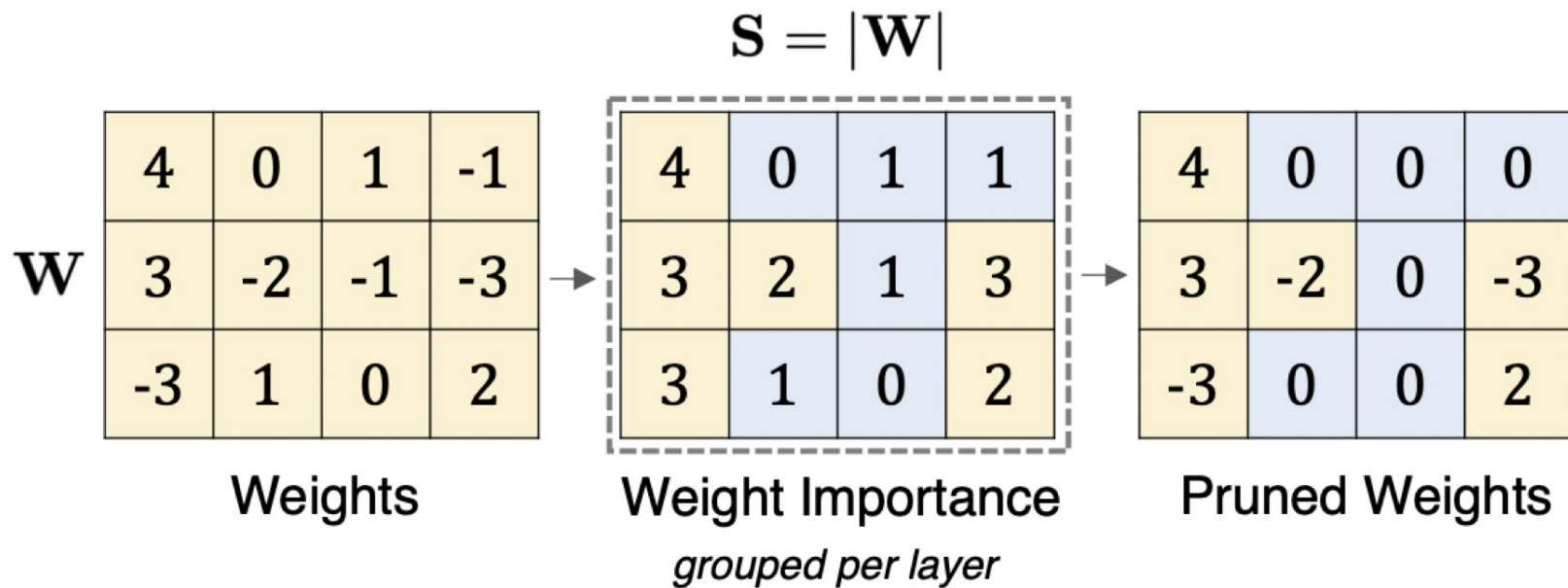
Q: What NN compression methods do you know?

Pruning

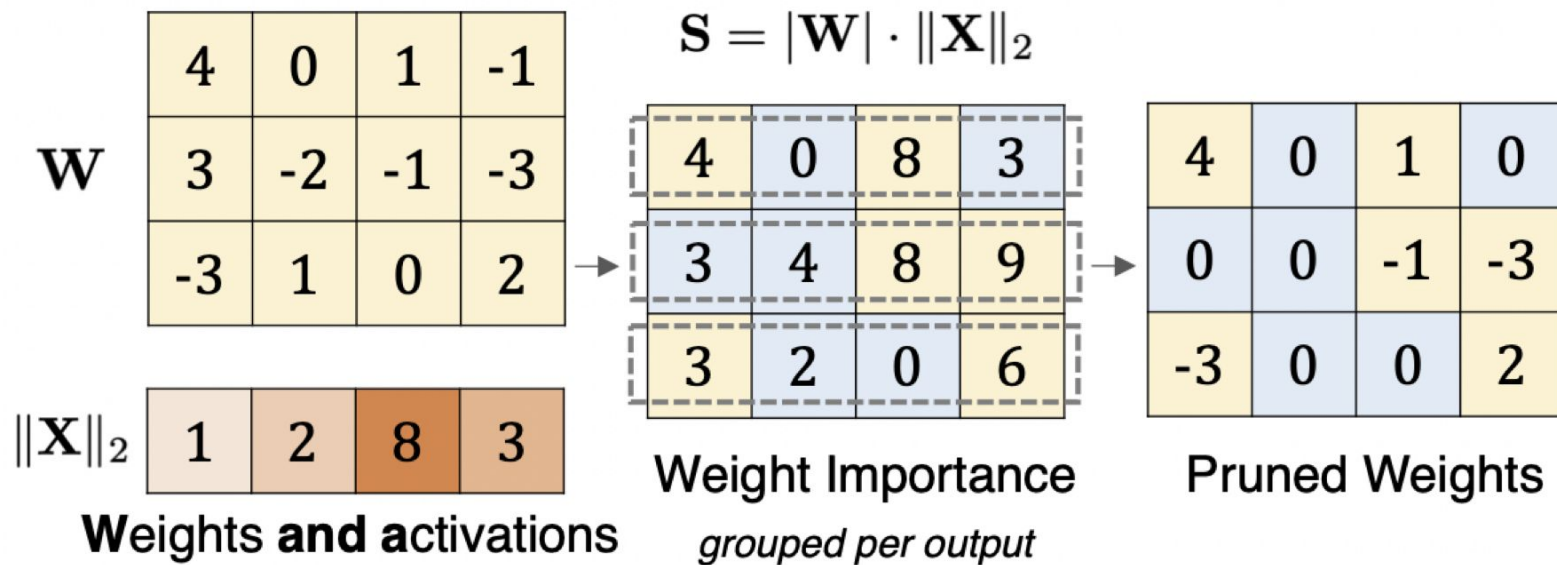


Pruning eliminates the weights/layers/channels, that does not contribute much to the final model performance.

Magnitude pruning



Wanda



SparseGPT

$$\operatorname{argmin}_{\text{mask } \mathbf{M}_\ell, \widehat{\mathbf{W}}_\ell} \|\mathbf{W}_\ell \mathbf{X}_\ell - (\mathbf{M}_\ell \odot \widehat{\mathbf{W}}_\ell) \mathbf{X}_\ell\|_2^2$$

$$\mathbf{S}_{ij} = \left[\frac{|\mathbf{W}|^2}{\operatorname{diag}\left((\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1}\right)} \right]_{ij}$$

Low-rank factorization

Recall that inference in a NN fundamentally entails calculating an affine transformation followed by a non-linearity

$$\sigma(Wx + b)$$

Low-rank factorization

$$\begin{aligned} W_{m \times n}^{(i)} &= U_{m \times m} \Sigma_{m \times n} V_{n \times n}^T = \\ &= \begin{bmatrix} u_{11} & \dots & u_{1m} \\ \vdots & \ddots & \vdots \\ u_{m1} & \dots & u_{mm} \end{bmatrix} \begin{bmatrix} \sigma_{11} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_{mn} \end{bmatrix} \begin{bmatrix} v_{11} & \dots & v_{1n} \\ \vdots & \ddots & \vdots \\ v_{n1} & \dots & v_{nn} \end{bmatrix} \\ &\approx \begin{bmatrix} u_{11} & \dots & u_{1k} \\ \vdots & \ddots & \vdots \\ u_{m1} & \dots & u_{mk} \end{bmatrix} \begin{bmatrix} \sigma_{11} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_{kk} \end{bmatrix} \begin{bmatrix} v_{11} & \dots & v_{1n} \\ \vdots & \ddots & \vdots \\ v_{k1} & \dots & v_{kn} \end{bmatrix}, \end{aligned}$$

where $k < \min(n, m)$

Low-rank factorization

$$W_{m \times n} = U_{m \times m} \Sigma_{m \times n} V_{n \times n}^T \approx \hat{U}_{m \times k} \hat{\Sigma}_{k \times k} \hat{V}_{k \times n}^T = \bar{U}_{m \times k} \hat{V}_{k \times n}^T$$

$$\sigma(W_{m \times n}x + b) \approx \sigma(\bar{U}_{m \times k} \hat{V}_{k \times n}^T x + b)$$

Instead of mn values, we store $k(m + n)$

If $k < \frac{mn}{m+n}$ we save space

e. g. if $m = n = 1000$, $k = 100$, then the number of trainable parameters in the current layer is reduced from 1000000 to 200000, a 80% reduction.

Low-rank factorization

$$W_{m \times n} = U_{m \times m} \Sigma_{m \times n} V_{n \times n}^T \approx \hat{U}_{m \times k} \hat{\Sigma}_{k \times k} \hat{V}_{k \times n}^T = \bar{U}_{m \times k} \hat{V}_{k \times n}^T$$

$$\sigma(W_{m \times n}x + b) \approx \sigma(\bar{U}_{m \times k} \hat{V}_{k \times n}^T x + b)$$

Instead of mn values, we store $k(m + n)$

If $k < \frac{mn}{m+n}$ we save space

e. g. if $m = n = 1000$, $k = 100$, then the number of trainable parameters in the current layer is reduced from 1000000 to 200000, a 80% reduction.

Low-rank factorization

$$W_{m \times n} = U_{m \times m} \Sigma_{m \times n} V_{n \times n}^T \approx \hat{U}_{m \times k} \hat{\Sigma}_{k \times k} \hat{V}_{k \times n}^T = \bar{U}_{m \times k} \hat{V}_{k \times n}^T$$

$$\sigma(W_{m \times n} x + b) \approx \sigma(\bar{U}_{m \times k} \hat{V}_{k \times n}^T x + b)$$

Instead of mn values, we store $k(m + n)$

If $k < \frac{mn}{m+n}$ we save space

e. g. if $m = n = 1000$, $k = 100$, then the number of trainable parameters in the current layer is reduced from 1000000 to 200000, a 80% reduction.

Low-rank factorization

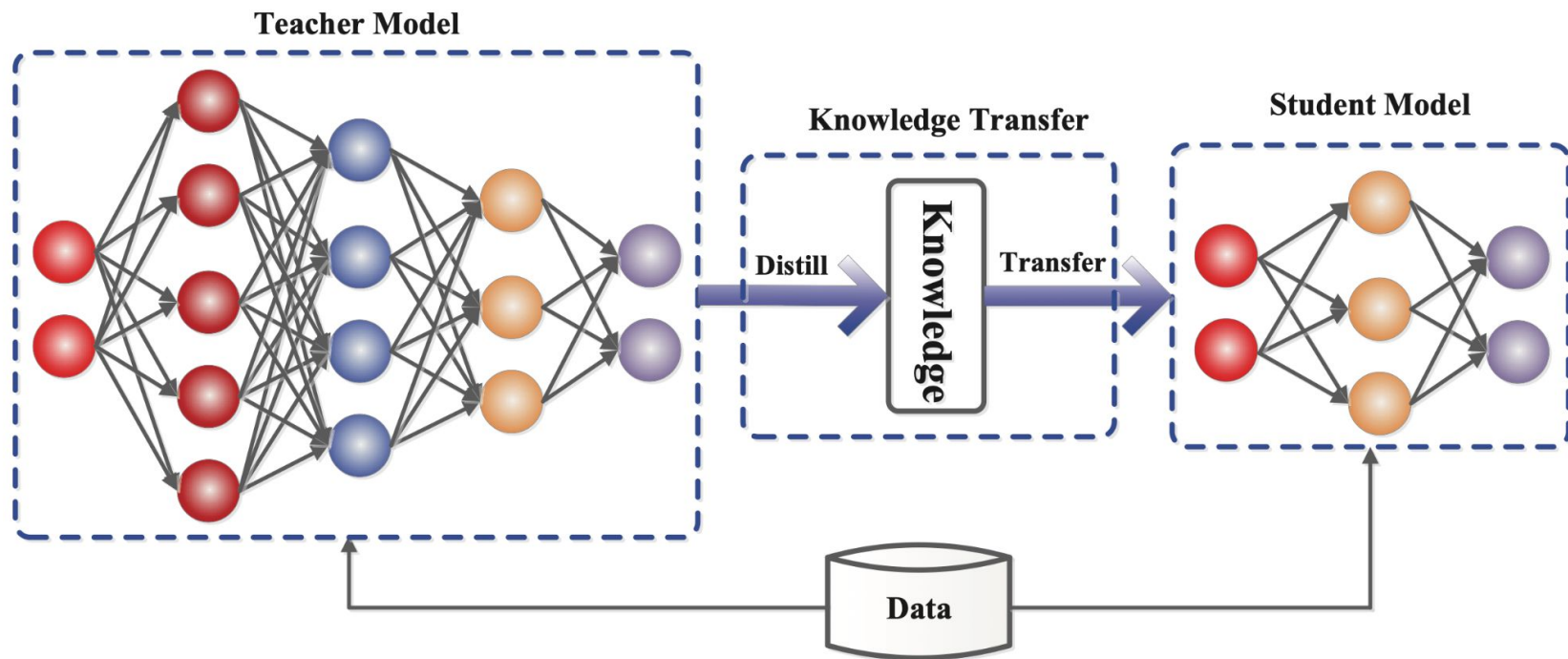
HOSVD:

$$\mathcal{A} = \mathcal{S} \times (\mathbf{U}_1, \mathbf{U}_2, \dots, \mathbf{U}_M).$$

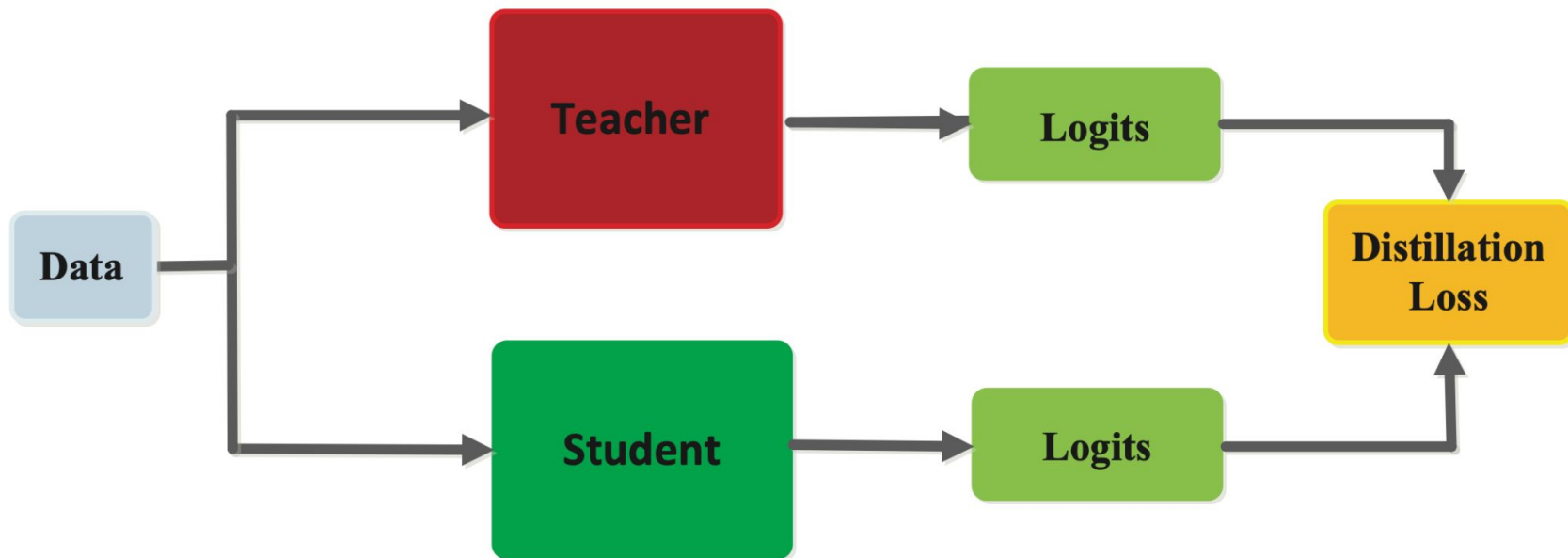
Tensor Train:

$$A(i_1, i_2, \dots, i_d) = \sum_{\alpha=1}^r U_1(i_1, \alpha) U_2(i_2, \alpha) \dots U_d(i_d, \alpha).$$

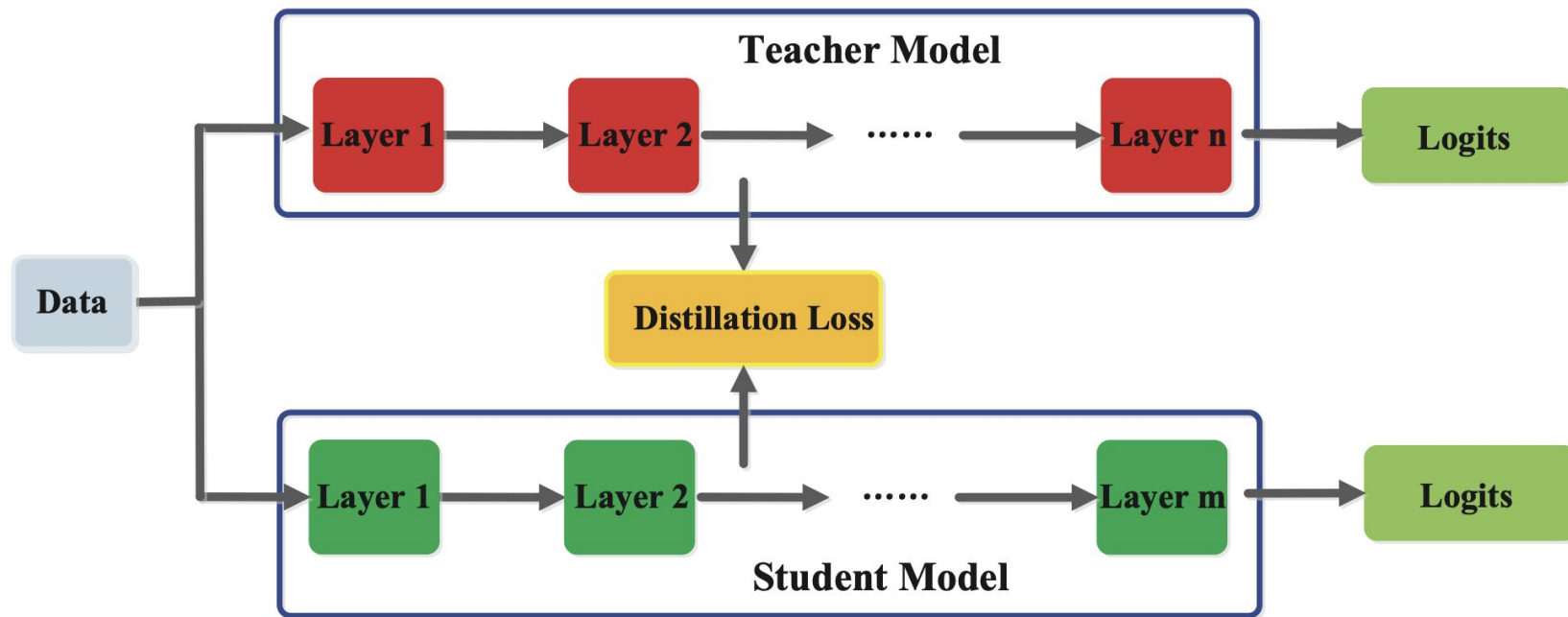
Knowledge distillation



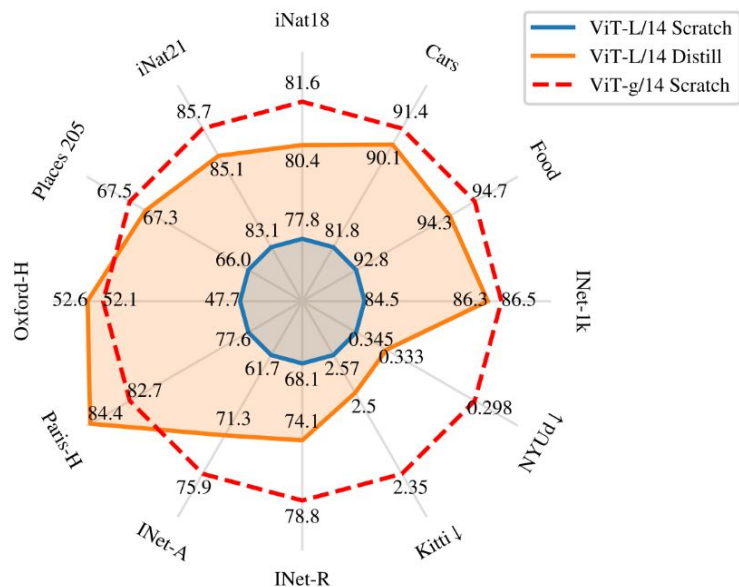
Response-based Knowledge Distillation



Feature-based Knowledge Distillation



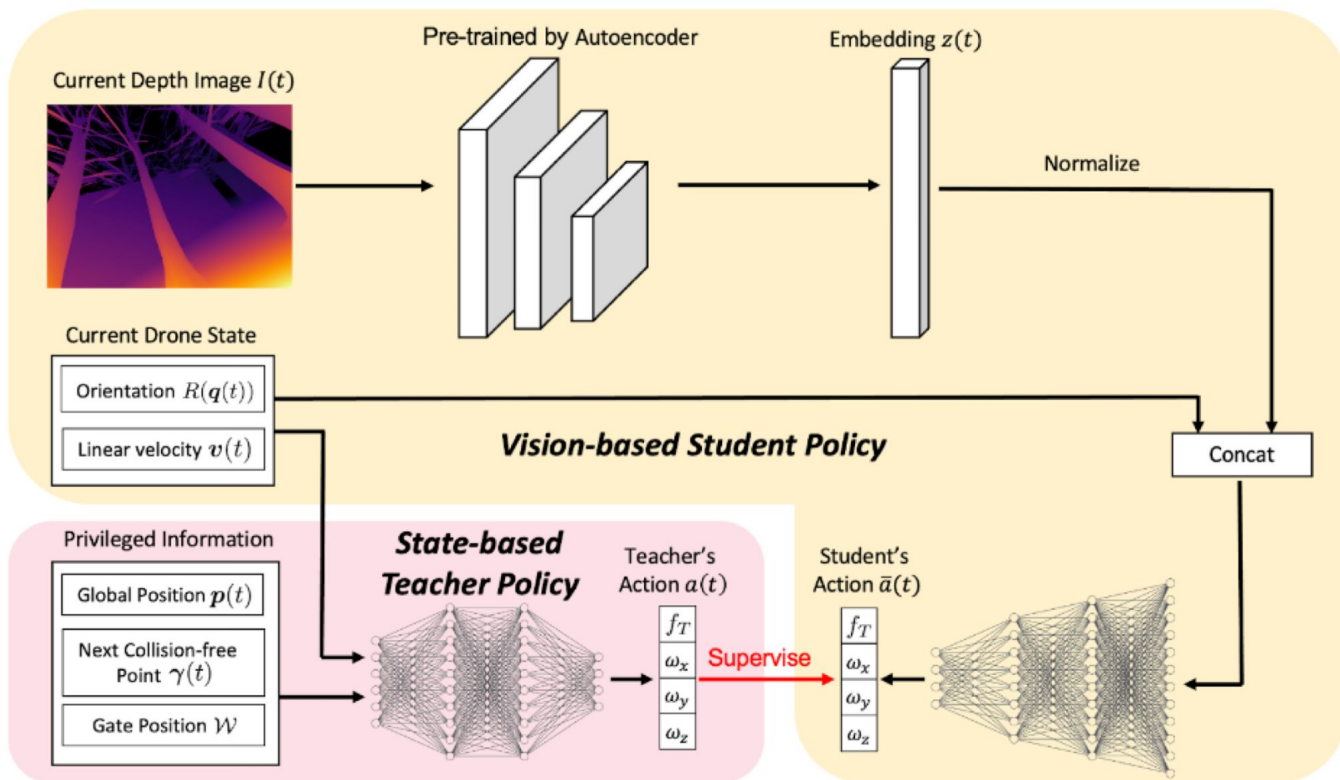
DINOv2 knowledge distillation



Arch	Method	INet-1k	Segm.	Depth↓	Classif.
ViT-g/14	Scratch	86.5	73.4	1.00	92.1
ViT-L/14	Scratch	84.5	72.2	1.10	90.2
ViT-L/14	Distill	86.3	73.3	1.08	91.2

Arch	Method	Finegr.	Retriev.	ARSketch	Video
ViT-g/14	Scratch	78.3	75.2	77.0	69.3
ViT-L/14	Scratch	75.8	71.3	69.5	67.3
ViT-L/14	Distill	77.6	76.3	74.5	67.5

Drone control policy knowledge distillation



Q: What is quantization?

Types of Quantization

1. Uniform scaling quantization
2. Rotation-enhanced uniform quantization
3. Non-uniform / codebook quantization
4. Calibration-based quantization
5. Quantization-Aware Training (QAT)

Types of Quantization

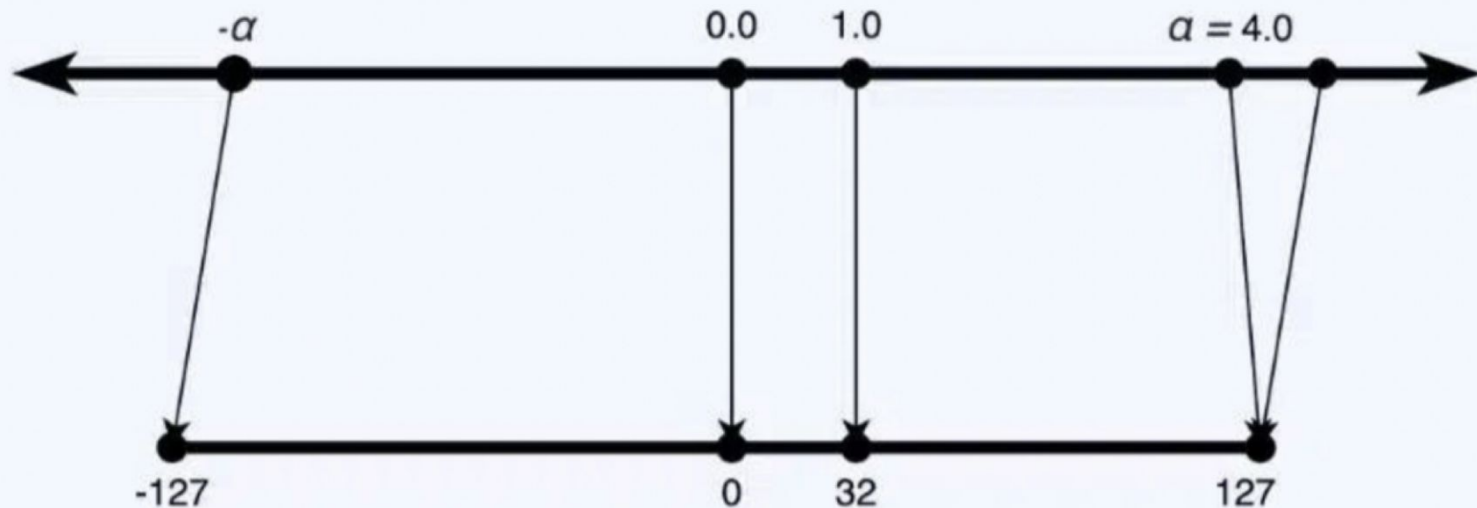
1. Uniform scaling quantization
2. Rotation-enhanced uniform quantization
3. Non-uniform / codebook quantization
4. Calibration-based quantization
5. Quantization-Aware Training (QAT)

Post Training Quantization (PTQ)

Uniform scaling quantization

The lower bound of larger precision

The upper bound of larger precision



Lower bound of int8

Scaling 1.0

Upper bound of int8

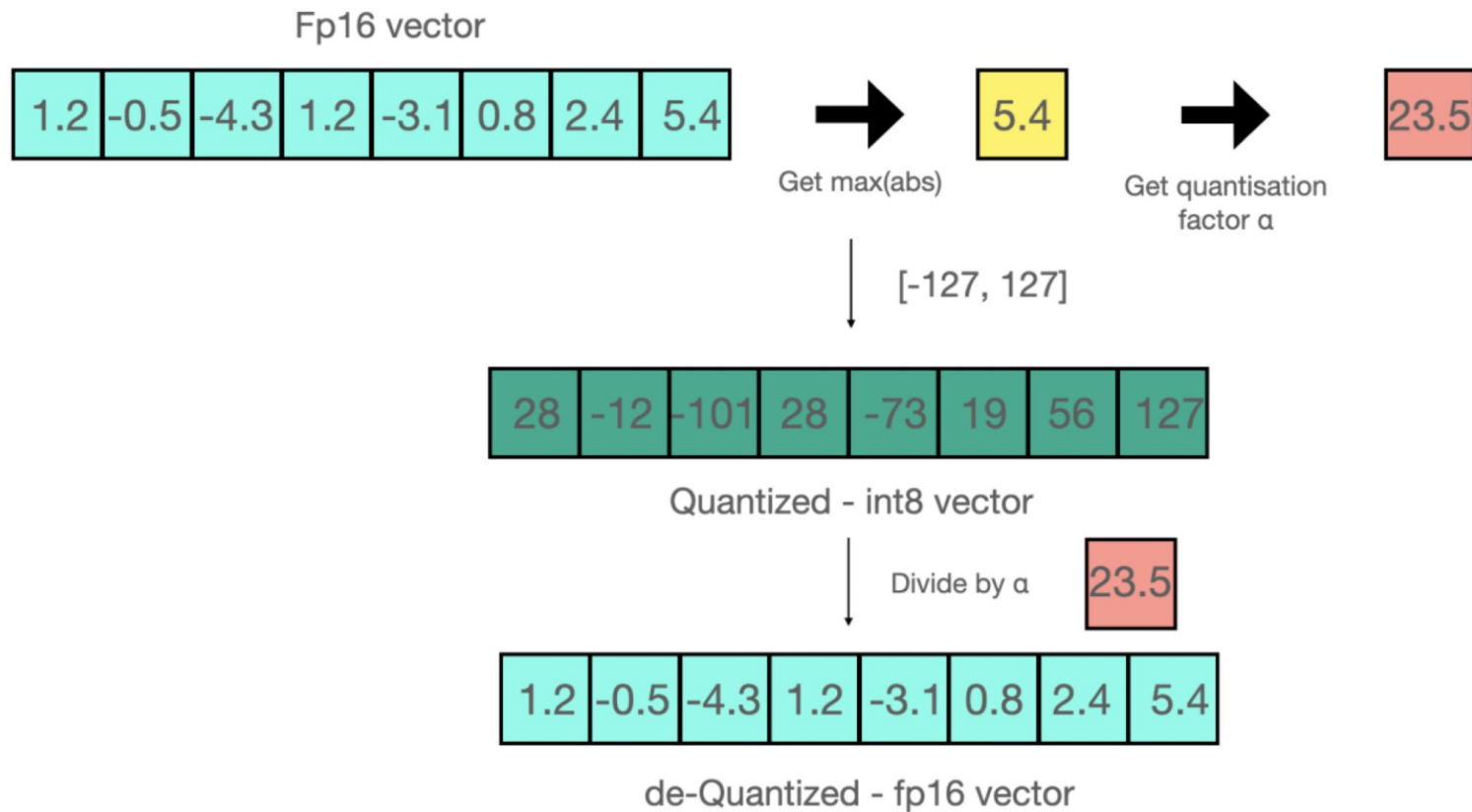
Uniform scaling quantization

$$X^{\text{int8}} = \text{round} \left(\frac{127}{\text{absmax}(X^{\text{FP32}})} X^{\text{FP32}} \right) = \text{round}(c^{\text{FP32}} \cdot X^{\text{FP32}}),$$

where c^{FP32} is *quantization constant* or *quantization scale*.

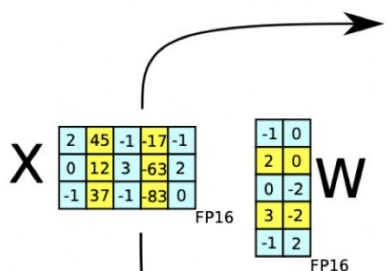
$$\text{dequant}(c^{\text{FP32}}, X^{\text{int8}}) = \frac{X^{\text{int8}}}{c^{\text{FP32}}} \approx X^{\text{FP32}}$$

Uniform scaling quantization



Uniform scaling quantization: handling outliers

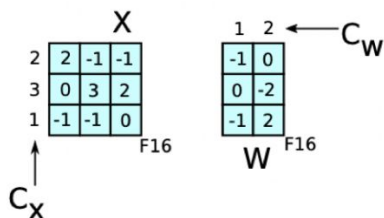
LLM.int8()



Regular values
Outliers

8-bit Vector-wise Quantization

(1) Find vector-wise constants: C_W & C_X



(2) Quantize

$$X_{F16} * (127/C_X) = X_{I8}$$

$$W_{F16} * (127/C_W) = W_{I8}$$

(3) Int8 Matmul

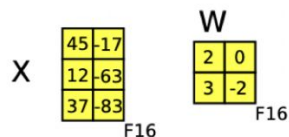
$$X_{I8} W_{I8} = Out_{I32}$$

(4) Dequantize

$$\frac{Out_{I32} * (C_X \otimes C_W)}{127 * 127} = Out_{F16}$$

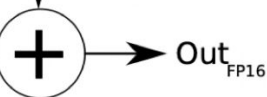
16-bit Decomposition

(1) Decompose outliers



(2) FP16 Matmul

$$X_{F16} W_{F16} = Out_{F16}$$



Uniform scaling quantization: llama.cpp

1. Legacy quants (Q4_0, Q4_1, Q8_0, ...)
 - a. blocks of 256 weights, and each block is turned into 256 quantized values and one (_0) or two (_1) extra constants
2. K-quants (Q3_K_S, Q5_K_M, ...)
 - a. Super-blocks containing 8-16 blocks, each block having 16-32 weights
 - b. Block scales and mins are quantized with 4-6 bits

Rotation-enhanced uniform quantization

- Compute rotation R (Hadamard, Householder, SVD, learned)
- Apply: $W' = WR$
- Uniform quantize W'
- Undo rotation

llama.cpp: I-quants (IQ2_XXS, IQ3_S, ...)

Non-uniform / codebook quantization

For a weight matrix $W \in \mathbf{R}^{m \times n}$, corresponding to a dense layer, we perform k-means clustering:

$$\min \sum_{i=1}^{m \cdot n} \sum_{j=1}^k \|w_i - c_j\|_2^2,$$

where w_i and c_j are both scalars

During prediction, we directly “look up” the values for each W_{ij} . This gives the reconstructed matrix:

$$\overline{W}_{ij} = c_z, \text{ where } \min_z \|W_{ij} - c_z\|_2^2$$

The general compression rate for this method is $32/\log_2(k)$, where k is the number of clusters.

Calibration-based quantization: GPTQ

GPTQ greedily quantizes weights one-by-one, choosing each quantized value to minimize activation MSE using a local Hessian approximation, and correcting the remaining weights after each step.

Global objective (NP-hard):

$$\hat{W}_l = \underset{\hat{W}_l}{\operatorname{argmin}} \|W_l X - \hat{W}_l X\|_2^2,$$

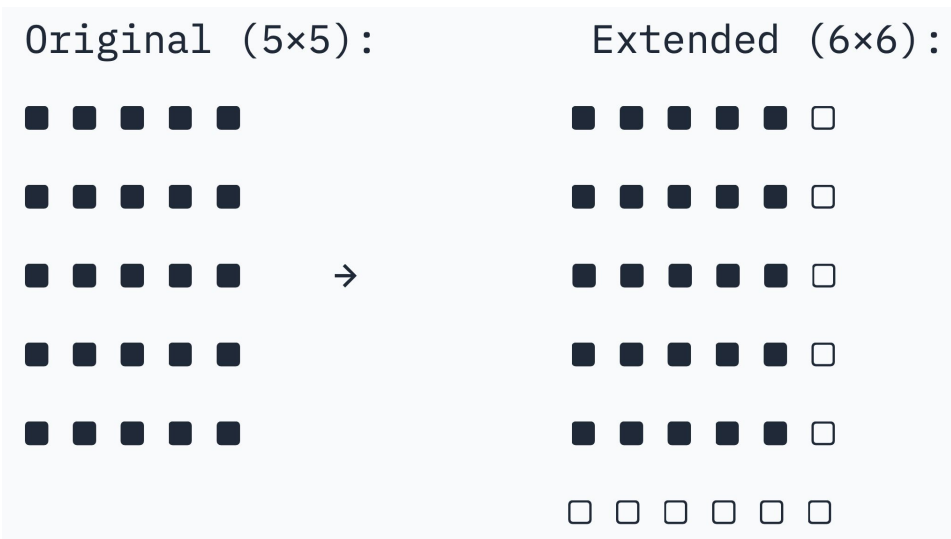
local approximation:

$$q_i = \underset{q \in \mathcal{Q}}{\operatorname{argmin}} \|X(w_i - q)\|^2$$

Inference

Key-Value (KV) Caching

In autoregressive generation, the model generates one token at a time. With each step, it recomputes Q , K , and V for **the entire sequence**, even though the earlier tokens haven't changed.



Key-Value (KV) Caching

$$Q_t = h_t W_Q$$

$$K_t = h_t W_K$$

$$V_t = h_t W_V$$

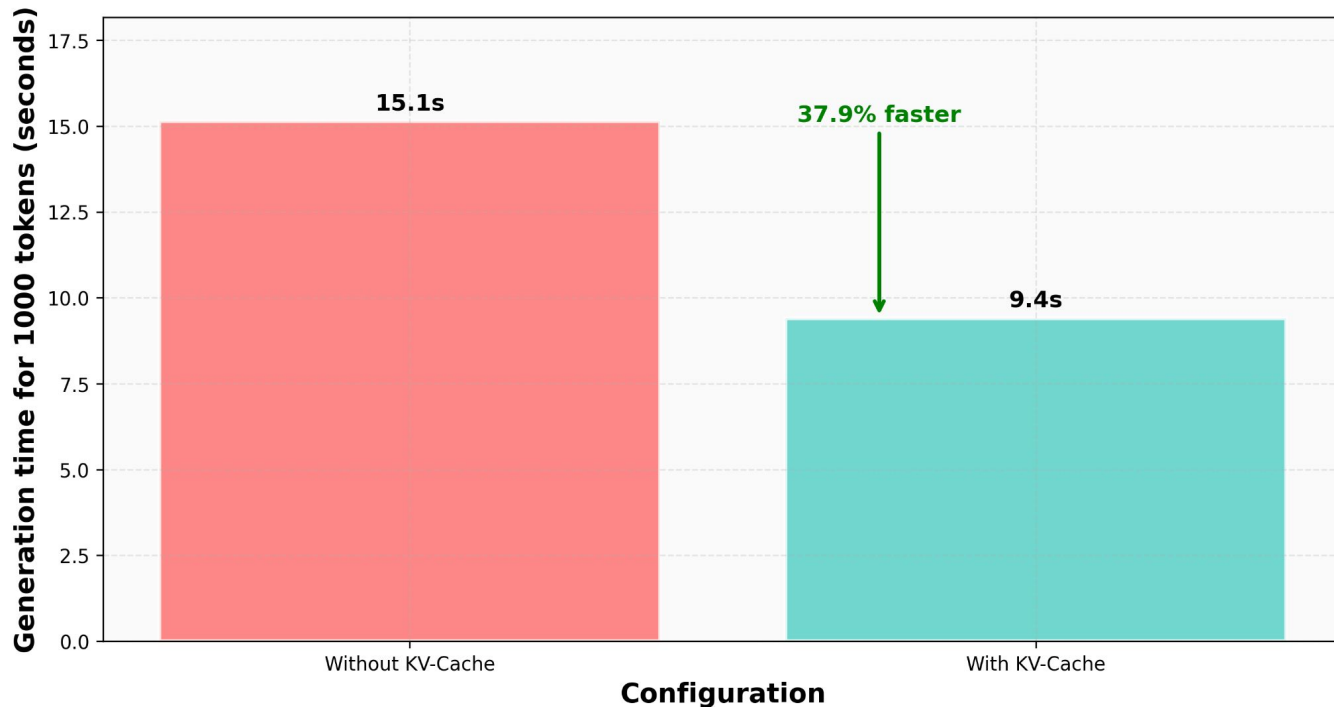
$$K_{1:t} = [K_{1:t-1}; K_t]$$

$$V_{1:t} = [V_{1:t-1}; V_t]$$

$$\text{Attn}(Q_t, K_{1:t}, V_{1:t}) = \text{softmax}\left(\frac{Q_t K_{1:t}^\top}{\sqrt{d}}\right) V_{1:t}$$

Key-Value (KV) Caching

**LLM Generation Speed: KV-Cache Performance Comparison
(Lower is Better)**

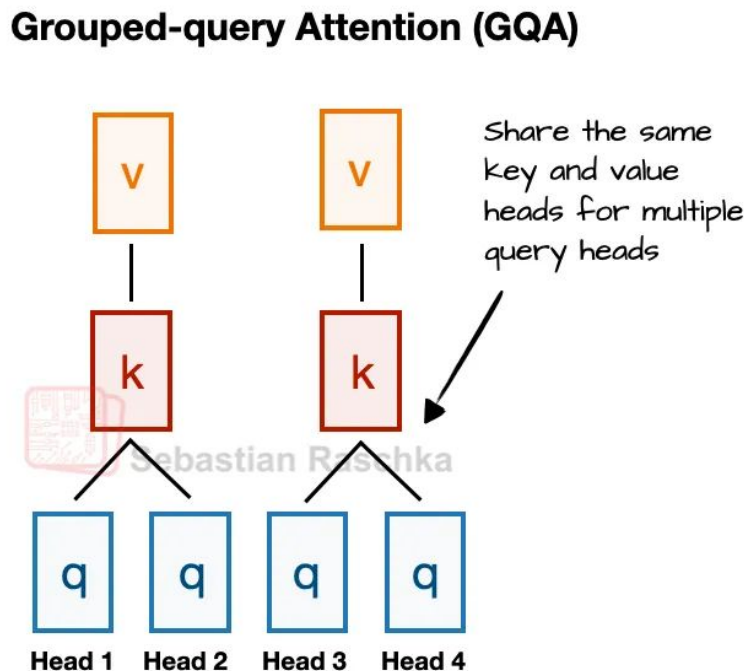
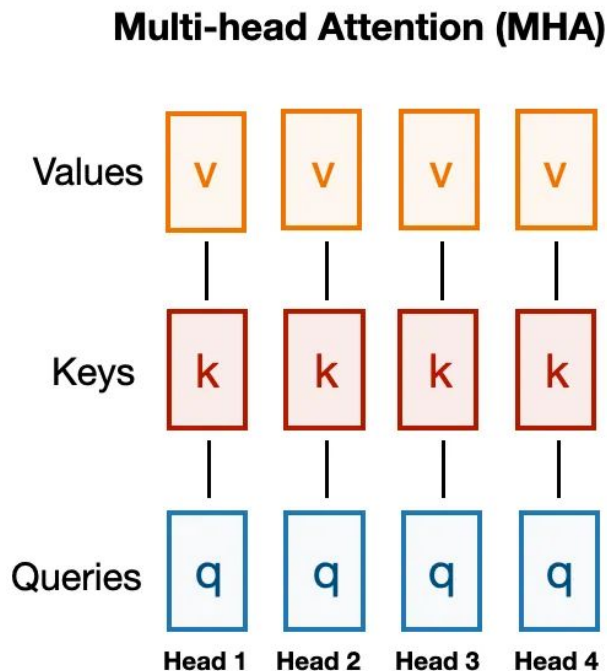


Prefix Caching

By caching the KV cache of an existing query, a new query that shares the same prefix can skip recomputing that part of the prompt. Instead, it directly reuses the cached results.

1. During prefill, the model performs a forward pass over the entire input and builds up a key-value (KV) cache for attention computation.
2. During decode, the model generates output tokens one by one, using the cached states from the prefill stage. The attention mechanism computes a matrix of token interactions. The resulting KV pairs for each token are stored in GPU memory.
3. For a new request with a matching prefix, you can skip the forward pass for the cached part and directly resume from the last token of the prefix.

Grouped-Query Attention (GQA)



Grouped-Query Attention (GQA)

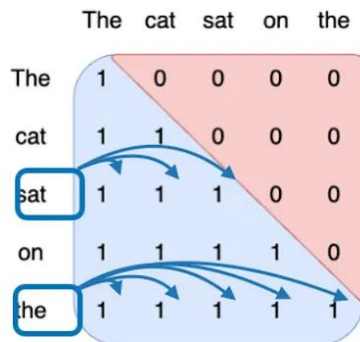
$$\text{head}_i = \text{Attention}(X_Q W_i^Q, X_K W_{g(i)}^K, X_V W_{g(i)}^V)$$

$$\text{GQA}(X_Q, X_K, X_V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

- GQA: share K/V across groups of heads.
- Cuts KV memory by **8x–16x**.
- Enables larger batch sizes for inference engines.

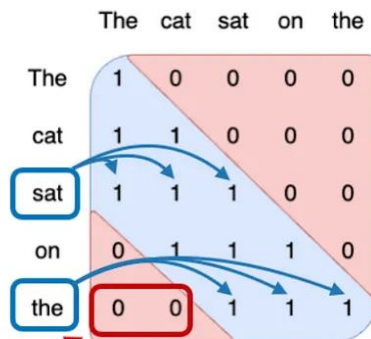
Sliding Window Attention

Regular causal self-attention mask



Using a causal attention mask, the current token can only attend previous tokens (+ itself)

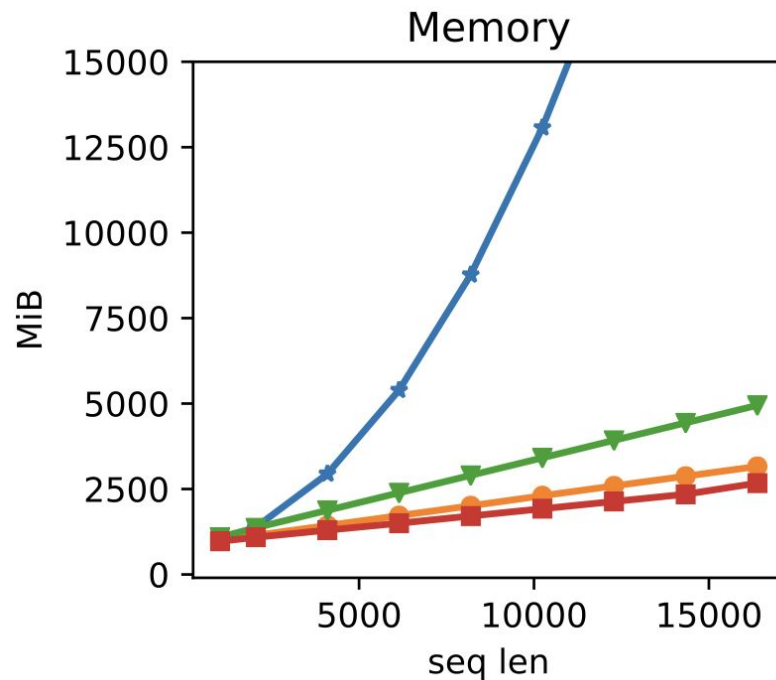
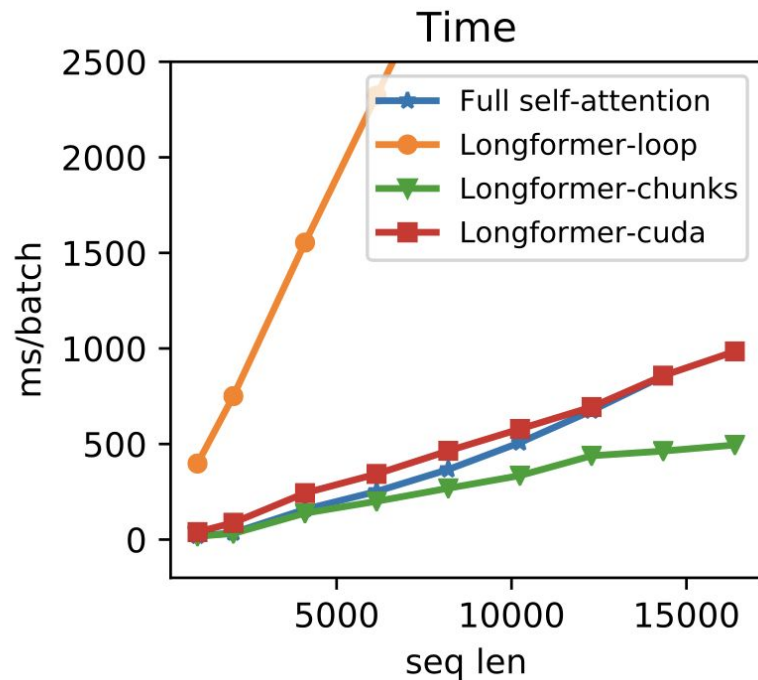
Sliding window attention



Not attended to save computation

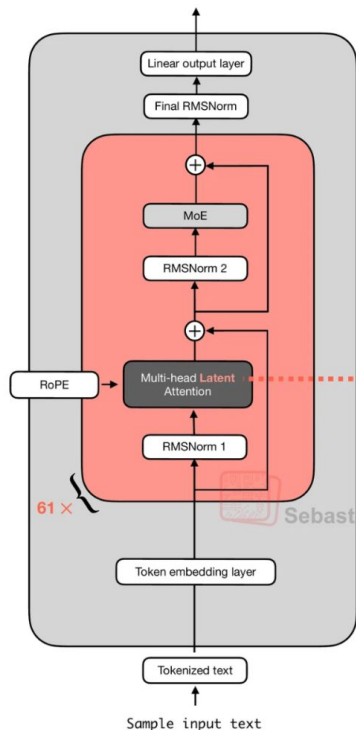
Using a causal attention mask, the current token can only attend previous tokens **within a certain limit**

Sliding Window Attention



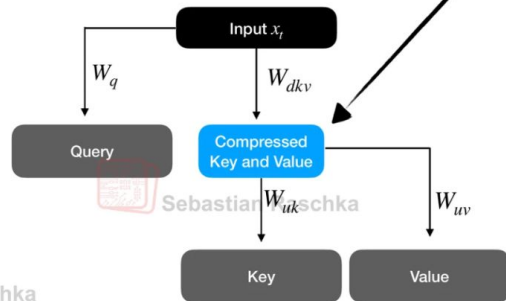
Multi-head Latent Attention (MLA)

DeepSeek V3/R1



Multi-head Latent Attention (MLA)

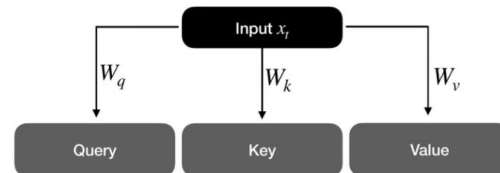
Inference step t :



Key idea: Reduces memory usage in KV cache

Regular Multi-head Attention (MHA)

Inference step t :



Multi-head Latent Attention (MLA)

For input sequence X , self-attention using MLA computes:

$$\begin{aligned}Q &= XW_Q^D W_Q^U = (XW_Q^D)W_Q^U = C_Q W_Q^U \\K &= XW_{KV}^D W_K^U = (XW_{KV}^D)W_K^U = C_{KV} W_K^U \\V &= XW_{KV}^D W_V^U = (XW_{KV}^D)W_V^U = C_{KV} W_V^U\end{aligned}$$

Where:

- $W_Q^D, W_{KV}^D \in \mathbb{R}^{d \times r}$ are low-rank compression matrices, with a small r , to reduce the dimension
- $W_Q^U, W_K^U, W_V^U \in \mathbb{R}^{r \times (n_h d_h)}$ are decompression matrices, to recover the dimension
- r is the latent dimension, typically $r \ll n_h \cdot d_h$

Multi-head Latent Attention (MLA)

$$\begin{aligned} O_i &= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \\ &= \text{softmax}\left(\frac{(XW_Q^D W_{Q,i}^U)(XW_{KV}^D W_{K,i}^U)^\top}{\sqrt{d_k}}\right)XW_{KV}^D W_V^U \\ &= \text{softmax}\left(\frac{XW_Q^D W_{Q,i}^U W_{K,i}^{U\top} W_{KV}^{D\top} X^\top}{\sqrt{d_k}}\right)XW_{KV}^D W_{V,i}^U \\ &= \text{softmax}\left(\frac{C_Q W_{Q,i}^U W_{K,i}^{U\top} C_{KV}^\top}{\sqrt{d_k}}\right)C_{KV} W_{V,i}^U \end{aligned}$$

$$W_{QK,i} = W_{Q,i}^U W_{K,i}^{U\top}$$

- shared factor C_{KV} for both the K and V projections
- both C_Q and C_{KV} are computed once and shared across all heads
- $W_{QK,i}$ pre-computed and cached, saving time during inference

Unlike what previous papers found, this paper finds that MHA performs much BETTER than GQA

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

Table 9 | Comparison between MLA and MHA on hard benchmarks. DeepSeek V2 shows better performance than MHA, but requires a significantly smaller amount of KV cache.

The memory requirements for MLA are much lower than for MHA

MLA performs better than MHA (here tested on Mixture-of-Experts architectures)

Key-Value (KV) Caching

